NexusGear

Memoria del Proyecto – EPD 3: Frameworks – Laravel

Tecnologías Avanzadas de Desarrollo
Grado en Ingeniería Informática en Sistemas de Información
Curso 2025–2026

Índice

1. 1. Descripción del Proyecto
2. 2. Arquitectura de la Base de Datos (UML)
3. 3. Objetivos y Requisitos
4. 4. Casos de Uso
5. 5. Decisiones de Diseño
6. 6. Estado Actual de la Implementación
7. 7. Estructura del Código
8. 8. Rutas del Sistema
9. 9. Seeders y Datos de Prueba
10. 10. Configuración SMTP y Correo
11. 11. Control de Versiones y Kanban
12. 12. Pendiente / Trabajo Futuro

1. Descripción del Proyecto

NexusGear es un comercio electrónico especializado en periféricos tecnológicos de alta calidad, orientado a dos perfiles claramente diferenciados: el usuario Gamer Pro y el usuario Office & Focus. El proyecto ha sido desarrollado sobre Laravel 12 con el paquete de autenticación Fortify, integrando Bootstrap 5 para las interfaces y Eloquent ORM para la capa de datos.

La paleta cromática principal del proyecto gira en torno al azul marino (#1F497D), color elegido por su psicología de confianza, tecnología y profesionalidad, coherente con la propuesta de valor de la marca.

Tecnologías principales:

- Laravel 12 (framework PHP)
- Laravel Fortify 1.36 (autenticación)
- Bootstrap 5 (frontend / diseño responsive)
- Eloquent ORM (modelos y relaciones)
- Blade (sistema de plantillas)
- MySQL (base de datos relacional)
- Vite + SASS (compilación de assets)
- Mailtrap / SMTP (notificaciones por correo)
- PHP 8.3+

2. Arquitectura de la Base de Datos (UML)

El diseño relacional cubre todas las entidades necesarias para un e-commerce completo. A continuación se describen las tablas, sus campos principales y las relaciones entre ellas.

2.1 Tablas y Campos

Tabla	Campos	Descripción
users	id, name, email, password, email_verified_at, remember_token, timestamps	Usuarios registrados. Cada usuario tiene exactamente un carrito (1:1).
roles	id, nombre_rol	Roles del sistema: admin y customer.
rol_usuario	usuario_id (FK), rol_id (FK), asignado_el (datetime)	Tabla pivote N:M entre usuarios y roles. Clave primaria compuesta.
productos	id, nombre, precio (decimal), descripcion (text), stock (int), perfil (enum: gamer/office), destacado (bool), timestamps	Catálogo de productos.
categorias	id, nombre	Categorías de productos.
producto_categoria	producto_id (FK), categoria_id (FK)	Tabla pivote N:M entre productos y categorías. Clave primaria compuesta.
carritos	id, usuario_id (FK unique), timestamps	Un carrito por usuario (relación 1:1 con users).
item_carrito	carrito_id (FK), producto_id (FK), cantidad (int), precio_actual (decimal)	Líneas del carrito. Clave primaria compuesta. Almacena el precio en el momento de añadir.
pedidos	id, usuario_id (FK), estado (enum: pendiente/procesando/enviado/entregado/cancelado), fecha (datetime)	Pedidos realizados por los usuarios.
linea_pedido	pedido_id (FK), producto_id (FK), cantidad (int), precio_unitario (decimal), subtotal (decimal)	Líneas de pedido. Clave primaria compuesta. Precio congelado en el momento de la compra.
facturas	id, pedido_id (FK unique), numero_factura (unique), subtotal (decimal), iva (decimal), total_factura (decimal), fecha_emision (datetime)	Factura generada por cada pedido. Relación 1:1 con pedidos. IVA al 21%.

direcciones	id, usuario_id (FK), calle, numero, ciudad, codigo_postal	Direcciones de envío del usuario (1:N con users).
favoritos	usuario_id (FK), producto_id (FK), agregado_el (datetime)	Lista de favoritos. Clave primaria compuesta N:M entre users y productos.
descuentos	id, codigo (unique), tipo, valor, fecha_fin (datetime)	Cupones de descuento.
descuento_producto	descuento_id (FK), producto_id (FK)	Tabla pivote N:M entre descuentos y productos.

2.2 Cardinalidades

El esquema cumple con el requisito de incluir al menos una relación de cada cardinalidad:

- 1:1 → users ↔ carritos (cada usuario tiene exactamente un carrito)
- 1:1 → pedidos ↔ facturas (cada pedido genera exactamente una factura)
- 1:N → users → pedidos (un usuario puede tener muchos pedidos)
- 1:N → users → direcciones (un usuario puede tener varias direcciones)
- 1:N → carritos → item_carrito (un carrito contiene varios ítems)
- 1:N → pedidos → linea_pedido (un pedido tiene varias líneas)
- N:M → users ↔ roles (vía rol_usuario)
- N:M → productos ↔ categorias (vía producto_categoria)
- N:M → users ↔ productos (favoritos, vía favoritos)
- N:M → descuentos ↔ productos (vía descuento_producto)

3. Objetivos y Requisitos

3.1 Objetivo General

Desarrollar un e-commerce completo y funcional para la venta de periféricos tecnológicos, implementando buenas prácticas de desarrollo web con el framework Laravel y respetando los principios del patrón MVC.

3.2 Objetivos Específicos

- Implementar operaciones CRUD completas con PHP–Laravel integrando Bootstrap 5.
- Diseñar y desarrollar el caso de uso principal: compra de productos por parte de usuarios registrados.
- Integrar un sistema de Login–Signin completo con verificación de correo electrónico (Laravel Fortify).
- Diseñar y desarrollar un panel de administración con gestión de productos y pedidos.
- Configurar el protocolo SMTP con Laravel para el envío de correos de confirmación de pedidos.
- Gestionar las categorías de productos (CRUD para administradores, filtrado para todos los usuarios).
- Implementar una lista de favoritos por usuario.
- Gestionar el perfil de usuario: contraseña, direcciones y pedidos.
- Internacionalizar la aplicación (ES/EN).
- Mantener el control de versiones con Git y GitHub con tablero Kanban.

3.3 Requisitos Funcionales

Versión 1.0 (Caso de uso principal):

- El sistema debe permitir la navegación del catálogo a usuarios no registrados.
- Se exigirá registro/inicio de sesión al tramitar un pedido o usar la lista de favoritos.
- Los usuarios registrados deben poder añadir productos al carrito, modificar cantidades y vaciar el carrito.
- El proceso de checkout debe validar el stock disponible y actualizar el inventario de forma transaccional.
- Tras una compra exitosa, el sistema debe generar una factura con IVA (21%) y número único (formato NG-YYYYMMDD-ID).
- El usuario debe recibir un correo electrónico de confirmación con los detalles del pedido.
- Los usuarios registrados deben poder consultar el historial de sus pedidos y su estado.
- Un usuario estándar nunca puede acceder a las funcionalidades del administrador.
- El administrador debe poder realizar CRUD de productos.
- El administrador debe poder visualizar todos los pedidos y actualizar su estado.

Versión 2.0 (Categorías):

- Un producto puede pertenecer a una o varias categorías; una categoría puede tener cero o varios productos (N:M).
- El administrador gestiona las categorías (CRUD) y las asigna a los productos al crear/editar.
- Todos los usuarios pueden filtrar el catálogo por categoría.

Versión 2.1 (Traducciones y perfil):

- La aplicación debe estar disponible en español e inglés.
- El usuario puede cambiar el idioma desde su perfil.
- El perfil del usuario permite gestionar contraseña, direcciones (CRUD) y consultar pedidos.

Versión 2.2 (Favoritos):

- Cualquier producto puede agregarse o quitarse de favoritos con un solo clic.
- El perfil del usuario incluye una sección para ver sus favoritos.
- El administrador puede ver los productos más marcados como favoritos.

3.4 Requisitos No Funcionales

- Uso obligatorio de Bootstrap 5 para las interfaces (diseño responsive).
- La base de datos debe estar diseñada en UML antes de la implementación.
- El esquema relacional debe incluir al menos una relación 1:1, 1:N y N:M.
- Uso de seeders para poblar la base de datos con datos de prueba.
- Control de transacciones en operaciones críticas (checkout).
- Código gestionado con Git en repositorio público en GitHub.
- Gestión del flujo de trabajo mediante tableros Kanban.
- Documentación activa de decisiones de diseño importantes.
- El sistema debe ser funcional excepto la pasarela de pago real.

4. Casos de Uso

CU-01: Navegar por el catálogo de productos

Actor principal: Cualquier visitante (registrado o no)

Precondición: Ninguna. El catálogo es público.

Flujo principal:

- 1. El visitante accede a la tienda y ve los productos disponibles.
- 2. Puede buscar productos por nombre o descripción.
- 3. Puede filtrar el catálogo por perfil (Gamer u Office) y por categoría.
- 4. Puede ordenar los resultados por precio o por nombre.
- 5. Al hacer clic en un producto, accede a su página de detalle, donde puede ver la descripción completa, el precio y el stock disponible.

CU-02: Registrarse en la plataforma

Actor principal: Visitante no registrado

Precondición: El visitante no tiene cuenta en NexusGear.

Postcondición: El usuario tiene una cuenta creada y recibe un correo de verificación.

Flujo principal:

- 1. El visitante accede al formulario de registro.
- 2. Introduce su nombre, correo electrónico y contraseña.
- 3. El sistema comprueba que el correo no esté ya en uso y que la contraseña cumpla los requisitos mínimos de seguridad.
- 4. Si los datos son válidos, el sistema crea la cuenta y envía un correo de verificación.
- 5. El usuario es redirigido a una pantalla que le indica que revise su bandeja de entrada.

Flujo alternativo – Datos inválidos:

- Si el correo ya existe o la contraseña no es válida, el sistema muestra los errores correspondientes y el visitante puede corregirlos.

CU-03: Verificar correo electrónico

Actor principal: Usuario recién registrado

Precondición: El usuario ha completado el registro pero aún no ha verificado su correo.

Postcondición: La cuenta queda verificada y el usuario puede comprar y usar favoritos.

Flujo principal:

- 1. El usuario abre el correo de verificación recibido tras el registro.
- 2. Hace clic en el enlace de verificación.

- 3. El sistema confirma que la cuenta es válida y marca el correo como verificado.
- 4. El usuario queda habilitado para realizar compras y gestionar favoritos.

Nota: Hasta que el correo no esté verificado, el usuario puede navegar por el catálogo pero no puede añadir productos al carrito ni marcar favoritos.

CU-04: Iniciar sesión

Actor principal: Usuario registrado

Precondición: El usuario tiene una cuenta creada en NexusGear.

Postcondición: El usuario accede a su cuenta y puede usar todas las funcionalidades disponibles para su rol.

Flujo principal:

- 1. El usuario accede al formulario de inicio de sesión.
- 2. Introduce su correo electrónico y contraseña.
- 3. El sistema verifica que las credenciales son correctas.
- 4. El usuario es redirigido a su página de inicio. Si tiene rol de administrador, accede al panel de administración.

Flujo alternativo – Credenciales incorrectas:

- Si el correo o la contraseña no coinciden, el sistema muestra un mensaje de error y el usuario puede volver a intentarlo.

CU-05: Cerrar sesión

Actor principal: Usuario registrado con sesión activa

Precondición: El usuario ha iniciado sesión.

Postcondición: La sesión queda cerrada y el usuario pasa a ser un visitante anónimo.

Flujo principal:

- 1. El usuario pulsa la opción de cerrar sesión disponible en el menú de navegación.
- 2. El sistema cierra la sesión de forma segura.
- 3. El usuario es redirigido a la página de inicio de la tienda.

CU-06: Recuperar contraseña olvidada

Actor principal: Usuario registrado

Precondición: El usuario tiene una cuenta pero no recuerda su contraseña.

Postcondición: El usuario establece una nueva contraseña y puede volver a iniciar sesión.

Flujo principal:

- 1. El usuario accede a la opción "¿Has olvidado tu contraseña?" en el formulario de inicio de sesión.
- 2. Introduce su correo electrónico.
- 3. El sistema envía un enlace de recuperación a ese correo.
- 4. El usuario abre el correo y hace clic en el enlace.
- 5. El sistema muestra un formulario para introducir una nueva contraseña.
- 6. El usuario introduce y confirma la nueva contraseña.
- 7. El sistema actualiza la contraseña y el usuario puede iniciar sesión con ella.

CU-07: Gestionar el carrito de compra

Actor principal: Usuario registrado y verificado

Precondición: El usuario ha iniciado sesión y tiene el correo verificado.

Flujo principal:

- 1. El usuario puede añadir cualquier producto disponible a su carrito desde el catálogo o desde la página de detalle. El sistema comprueba que hay stock suficiente antes de añadirlo.
- 2. Desde el carrito, el usuario puede cambiar la cantidad de cualquier artículo. Si la cantidad solicitada supera el stock, el sistema avisa y no aplica el cambio.
- 3. El usuario puede eliminar un artículo concreto de su carrito.
- 4. El usuario puede vaciar el carrito completo con un solo clic.
- 5. El carrito muestra en todo momento el precio total actualizado.

CU-08: Realizar una compra

Actor principal: Usuario registrado y verificado

Precondición: El usuario tiene al menos un producto en su carrito.

Postcondición: El pedido queda registrado, el stock se actualiza y el usuario recibe una confirmación por correo.

Flujo principal:

- 1. El usuario accede a su carrito y revisa los productos que va a comprar.
- 2. Pulsa "Finalizar compra".
- 3. El sistema verifica el stock disponible de cada artículo en ese momento.
- 4. Si todo está disponible, el sistema registra el pedido, descuenta las unidades del inventario y genera una factura con IVA incluido (21%).
- 5. El usuario recibe un correo electrónico con el resumen del pedido y la factura.
- 6. El usuario es redirigido a la página de su pedido, donde puede consultar el estado y los detalles.

Flujo alternativo – Sin stock suficiente:

- Si en el momento de confirmar la compra algún producto no tiene stock suficiente, la operación se cancela por completo: no se crea ningún pedido, no se modifica el inventario y el carrito permanece intacto.
- El usuario recibe un mensaje indicando qué producto causó el problema.

CU-09: Consultar historial de pedidos

Actor principal: Usuario registrado y verificado

Precondición: El usuario ha iniciado sesión.

Flujo principal:

- 1. El usuario accede a la sección de pedidos desde su menú de usuario.
- 2. Ve el listado de todos sus pedidos ordenados del más reciente al más antiguo, con el estado de cada uno.
- 3. Al hacer clic en un pedido, puede ver el detalle completo: productos comprados, cantidades, precios y datos de la factura.

CU-10: Gestionar favoritos

Actor principal: Usuario registrado y verificado

Precondición: El usuario ha iniciado sesión y tiene el correo verificado.

Flujo principal:

- 1. Desde el catálogo o la página de detalle de un producto, el usuario puede marcarlo como favorito con un solo clic.
- 2. Si el producto ya estaba marcado como favorito, el mismo clic lo elimina de la lista.
- 3. El usuario puede acceder a su lista de favoritos desde su perfil para ver todos los productos guardados.

CU-11: Gestionar perfil de usuario

Actor principal: Usuario registrado

Precondición: El usuario ha iniciado sesión.

Flujo principal:

- 1. El usuario accede a su perfil desde el menú de navegación.
- 2. Puede cambiar su contraseña actual por una nueva.
- 3. Puede gestionar sus direcciones de envío: añadir nuevas, editarlas o eliminarlas.
- 4. Puede cambiar el idioma de la aplicación entre español e inglés.
- 5. Puede consultar su lista de favoritos y el historial de pedidos desde el mismo perfil.

CU-12: Gestionar productos (Administrador)

Actor principal: Administrador

Precondición: El usuario ha iniciado sesión con una cuenta de administrador.

Flujo principal:

- 1. El administrador accede a la sección de productos del panel de administración.
- 2. Puede ver el listado completo de productos con filtros por nombre y perfil, y navegar entre páginas.
- 3. Puede crear un nuevo producto indicando nombre, precio, descripción, stock disponible, perfil (Gamer u Office), si es destacado, y asignarle una o varias categorías.
- 4. Puede editar cualquier producto existente modificando cualquiera de sus datos.
- 5. Puede eliminar un producto, siempre que no esté incluido en ningún pedido ya realizado.

CU-13: Gestionar categorías (Administrador)

Actor principal: Administrador

Precondición: El usuario ha iniciado sesión con una cuenta de administrador.

Flujo principal:

- 1. El administrador accede a la sección de categorías del panel de administración.
- 2. Puede ver el listado de todas las categorías existentes.
- 3. Puede crear nuevas categorías, editarlas o eliminarlas.
- 4. Al crear o editar un producto, puede asignarle una o varias categorías de las disponibles.

CU-14: Gestionar pedidos (Administrador)

Actor principal: Administrador

Precondición: El usuario ha iniciado sesión con una cuenta de administrador.

Flujo principal:

- 1. El administrador accede a la sección de pedidos del panel de administración.
- 2. Puede ver todos los pedidos del sistema y filtrarlos por estado (pendiente, procesando, enviado, entregado, cancelado).
- 3. Puede abrir el detalle de cualquier pedido para ver los productos comprados, cantidades, precios y datos del cliente.
- 4. Puede actualizar el estado de un pedido para reflejar su progreso (por ejemplo, marcar un pedido como "enviado" cuando ha salido del almacén).

CU-15: Consultar productos más añadidos a favoritos (Administrador)

Actor principal: Administrador

Precondición: El usuario ha iniciado sesión con una cuenta de administrador.

Flujo principal:

- 1. El administrador accede al panel de administración.
- 2. Puede consultar un ranking con los productos que más veces han sido añadidos a favoritos por los usuarios.
- 3. Esta información le ayuda a identificar qué productos generan más interés entre los clientes.

5. Decisiones de Diseño

● Precio congelado en carrito e ítem de pedido

Tanto ItemCarrito como LineaPedido almacenan el precio en el momento de la operación (precio_actual y precio_unitario respectivamente). Esto garantiza que un cambio posterior en el precio de un producto no afecte pedidos ni carritos existentes.

● Transacción atómica en el checkout

Todo el proceso de compra (verificación de stock, creación de pedido, líneas de pedido, decremento de stock, vaciado de carrito y generación de factura) se ejecuta dentro de una única transacción DB::transaction(). Si cualquier paso falla, se ejecuta un rollback automático.

● Carrito creado al registrarse el usuario

Se usa un evento de Eloquent (Model::created en User o un Observer) para crear automáticamente el carrito del usuario en el momento de su registro, evitando comprobaciones posteriores de existencia.

● Claves primarias compuestas en tablas pivote

Las tablas item_carrito, linea_pedido, favoritos, rol_usuario, producto_categoria y descuento_producto no usan un ID autoincremental, sino que su clave primaria está formada por las dos claves ajenas. Esto garantiza la integridad referencial y evita duplicados a nivel de base de datos.

● Número de factura con formato único

El número de factura sigue el patrón NG-YYYYMMDD-{ID_PEDIDO}, lo que lo hace legible, ordenable cronológicamente y único por diseño.

● Middleware de roles

Se implementó el middleware CheckAdmin que verifica si el usuario autenticado posee el rol admin. Se aplica a todas las rutas del prefijo /admin, evitando que usuarios estándar accedan al panel.

● Accessors en modelos para formato europeo

Los modelos Producto, ItemCarrito, LineaPedido y Factura exponen accessors (precioFormateado, subtotalFormateado, etc.) que devuelven los valores monetarios con formato europeo (coma como separador decimal, símbolo €). Esto centraliza el formateo y mantiene las vistas limpias.

● Perfil de producto (Gamer / Office)

Cada producto tiene un atributo perfil con dos valores posibles: gamer y office. Esto permite filtrar el catálogo por tipo de usuario objetivo y mostrar contenido personalizado (icono de Bootstrap Icons según el nombre del producto).

● Bootstrap Icons para iconografía dinámica

El modelo Producto incluye el accessor icono que devuelve una clase de Bootstrap Icons según las palabras clave del nombre del producto (mouse, keyboard, pad, stand…), evitando almacenar imágenes en disco en esta versión.

● Seeders para datos de prueba

Se implementaron RolSeeder, ProductSeeder y UserSeeder ejecutados en orden desde DatabaseSeeder. Esto permite reconstruir un entorno de pruebas completo con php artisan migrate:fresh --seed. Credenciales de prueba: admin@nexusgear.com / admin123 y juan@example.com / user123.

6. Estado Actual de la Implementación

A continuación se resume el estado de cada versión/problema del proyecto a fecha 28/04/2026:

Problema / Versión	Funcionalidad	Estado
Problema 1	Esquema UML de base de datos	✅ Completado
Problema 2 – v1.0	Caso de uso principal: compra de productos	✅ Completado
Problema 2 – v1.0	Panel de administración (productos + pedidos)	✅ Completado
Problema 2 – v1.0	Sistema de autenticación completo (Fortify)	✅ Completado
Problema 2 – v1.0	Correo de confirmación de pedido (SMTP)	✅ Completado
Problema 2 – v1.0	Carrito de compra con gestión de stock	✅ Completado
Problema 2 – v1.0	Generación de facturas con IVA	✅ Completado
Problema 3 – v2.0	Modelo y migración de Categorias	✅ Completado
Problema 3 – v2.0	CRUD de categorías en admin	⏳ En progreso
Problema 3 – v2.0	Asignación de categorías al crear/editar producto	⏳ En progreso
Problema 3 – v2.0	Filtrado del catálogo por categoría	⏳ En progreso
Problema 4 – v2.1	Internacionalización ES/EN	❌ Pendiente
Problema 4 – v2.1	Gestión de perfil (contraseña, direcciones, idioma)	❌ Pendiente
Problema 5 – v2.2	Modelo y migración de Favoritos	✅ Completado
Problema 5 – v2.2	Toggle de favoritos con un clic	❌ Pendiente
Problema 5 – v2.2	Vista de favoritos en perfil de usuario	❌ Pendiente
Problema 5 – v2.2	Ranking de favoritos en panel de admin	❌ Pendiente

7. Estructura del Código

7.1 Controladores

ProductController

Ruta: app/Http/Controllers/ProductController.php

Métodos: index(), show()

Catálogo público. Filtra por búsqueda, perfil y ordenación; paginación de 9 productos.

CartController

Ruta: app/Http/Controllers/CartController.php

Métodos: index(), store(), update(), destroy(), clear()

Gestión del carrito de compra. Valida stock en cada operación.

OrderController

Ruta: app/Http/Controllers/OrderController.php

Métodos: index(), store(), show()

Checkout con transacción atómica, generación de factura y envío de correo. Historial de pedidos del usuario con paginación de 10.

Admin\ProductController

Ruta: app/Http/Controllers/Admin/Admin\ProductController.php

Métodos: index(), create(), store(), edit(), update(), destroy()

CRUD completo de productos en el panel admin. Filtra por nombre/perfil, paginación de 12.

Admin\OrderController

Ruta: app/Http/Controllers/Admin/Admin\OrderController.php

Métodos: index(), show(), update()

Listado de todos los pedidos con filtrado por estado y actualización de estado.

7.2 Modelos

User

Implementa MustVerifyEmail. Relaciones: roles (N:M), direcciones (1:N), pedidos (1:N), carrito (1:1), favoritos (1:N). Método isAdmin() para comprobar el rol.

Producto

Campos: nombre, precio, descripcion, stock, perfil, destacado. Accessors: precioFormateado, disponible, perfilNombre, icono. Relaciones: categorias (N:M), descuentos (N:M), favoritos (1:N), itemsCarrito (1:N), lineasPedido (1:N).

Carrito

Relación 1:1 con User. Accessors: total, totalFormateado, cantidadTotal. Relación 1:N con ItemCarrito.

ItemCarrito

Clave primaria compuesta (carrito_id, producto_id). Almacena precio_actual en el momento de añadir. Accessors: subtotal, subtotalFormateado.

Pedido

Estado: enum (pendiente/procesando/enviado/entregado/cancelado). Accessors: fechaFormateada, estadoLabel, estadoBadge (clase Bootstrap). Relaciones: usuario, factura (1:1), lineas (1:N).

LineaPedido

Clave primaria compuesta (pedido_id, producto_id). Almacena precio_unitario y subtotal congelados.

Factura

Relación 1:1 con Pedido. Campos: numero_factura (unique), subtotal, iva, total_factura. Accessors de formateo europeo.

Categoria

Relación N:M con Producto vía producto_categoria.

Rol
Relación N:M con User vía rol_usuario (pivot con campo asignado_el).

Direccion
Relación N:1 con User. Campos: calle, numero, ciudad, codigo_postal.

Favorito
Clave primaria compuesta (usuario_id, producto_id). Campo: agregado_el.

Descuento
Campos: codigo (unique), tipo, valor, fecha_fin. Relación N:M con Producto vía descuento_producto.

7.3 Vistas (Blade)

- layouts/app.blade.php — Layout principal con navbar, contador del carrito y menú de usuario.
- layouts/admin.blade.php — Layout del panel de administración.
- home.blade.php — Página principal con productos destacados.
- products/index.blade.php — Catálogo con filtros, búsqueda y paginación.
- products/show.blade.php — Detalle de producto con productos relacionados.
- cart/index.blade.php — Vista del carrito con resumen de totales.
- orders/index.blade.php — Historial de pedidos del usuario.
- orders/show.blade.php — Detalle de un pedido con líneas y factura.
- admin/dashboard.blade.php — Dashboard con KPIs (total productos, stock bajo, pedidos).
- admin/products/index.blade.php — Listado de productos para admin.
- admin/products/create.blade.php — Formulario de creación de producto.
- admin/products/edit.blade.php — Formulario de edición de producto.
- admin/products/_form.blade.php — Partial de formulario reutilizable.
- admin/orders/index.blade.php — Listado de todos los pedidos.
- admin/orders/show.blade.php — Detalle de pedido con cambio de estado.
- auth/login.blade.php — Formulario de inicio de sesión.
- auth/register.blade.php — Formulario de registro.
- auth/forgot-password.blade.php — Solicitud de recuperación de contraseña.
- auth/reset-password.blade.php — Restablecimiento de contraseña.
- auth/verify-email.blade.php — Verificación de correo electrónico.

- emails/orders/confirmation.blade.php — Plantilla del correo de confirmación de pedido.

7.4 Middleware

CheckAdmin (app/Http/Middleware/CheckAdmin.php): Verifica que el usuario autenticado posea el rol "admin". Si no, redirige a /home con un mensaje de error. Se registra como alias "admin" y se aplica a todas las rutas del grupo /admin.

8. Rutas del Sistema

Todas las rutas están definidas en routes/web.php.

8.1 Rutas Públicas (sin autenticación)

Método	URI	Descripción
GET	/	Página de inicio con productos destacados
GET	/productos	Catálogo de productos (búsqueda, perfil, orden)
GET	/productos/{producto}	Detalle de un producto

8.2 Rutas de Usuario Autenticado (middleware: auth, verified)

Método	URI	Descripción
GET	/home	Redirección al home o dashboard según el rol
GET	/carrito	Ver carrito
POST	/carrito/productos/{producto}	Añadir producto al carrito
PATCH	/carrito/productos/{producto}	Actualizar cantidad en carrito
DELETE	/carrito/productos/{producto}	Eliminar ítem del carrito
DELETE	/carrito	Vaciar carrito
POST	/checkout	Finalizar compra
GET	/pedidos	Ver historial de pedidos
GET	/pedidos/{pedido}	Ver detalle de pedido

8.3 Rutas de Administración (middleware: auth, verified, admin)

Método	URI	Descripción
GET	/admin/dashboard	Dashboard con KPIs
GET	/admin/products	Listado de productos
GET	/admin/products/create	Formulario de nuevo producto
POST	/admin/products	Guardar nuevo producto
GET	/admin/products/{producto}/edit	Formulario de edición
PATCH	/admin/products/{producto}	Actualizar producto
DELETE	/admin/products/{producto}	Eliminar producto
GET	/admin/orders	Listado de todos los pedidos
GET	/admin/orders/{pedido}	Detalle de pedido
PATCH	/admin/orders/{pedido}	Actualizar estado del pedido

9. Seeders y Datos de Prueba

La base de datos se puede poblar completamente con el comando php artisan migrate:fresh --seed. Los seeders se ejecutan en el siguiente orden:

RolSeeder

Inserta los roles "admin" y "customer".

ProductSeeder

Inserta 8 productos de muestra:
• Nexus Vertical Pro (59,99 €, office, destacado)
• Aqua Keys 60 (89,99 €, gamer, destacado)
• Focus Pad Gel (24,90 €, office, destacado)
• Split Core Ergo (129,00 €, office)
• Pulse Mouse X (74,50 €, gamer, destacado)
• Lift Dock Stand (39,95 €, office)
• Stealth Wrist Rest (29,99 €, gamer, stock 0)
• Tactile Flow TKL (109,90 €, gamer)

UserSeeder

Crea dos usuarios y les asigna roles:
• admin@nexusgear.com / admin123 → rol admin
• juan@example.com / user123 → rol customer

10. Configuración SMTP y Correo Electrónico

NexusGear envía correos electrónicos mediante la clase OrderConfirmationMail (app/Mail/OrderConfirmationMail.php) que extiende Mailable de Laravel.

Configuración requerida en .env:

- MAIL_MAILER=smtp
- MAIL_HOST=sandbox.smtp.mailtrap.io (o servidor SMTP real)
- MAIL_PORT=2525
- MAIL_USERNAME=<usuario_mailtrap>
- MAIL_PASSWORD=<password_mailtrap>
- MAIL_ENCRYPTION=tls
- MAIL_FROM_ADDRESS=noreply@nexusgear.com
- MAIL_FROM_NAME="NexusGear"

El correo se envía en el método store() de OrderController, tras confirmar la transacción de compra. El asunto sigue el formato: "Confirmación de pedido NG-YYYYMMDD-{ID}" y la plantilla emails/orders/confirmation.blade.php incluye el detalle del pedido, las líneas de productos y el total de la factura.

11. Control de Versiones y Kanban

El proyecto se gestiona con Git y está alojado en un repositorio público en GitHub. Se trabaja sobre la rama main con commits descriptivos. El tablero Kanban se mantiene en la sección Projects del repositorio, con las columnas: Backlog, En progreso, En revisión, Completado.

El repositorio incluye un .gitignore apropiado que excluye: el directorio /vendor, el directorio /node_modules, el archivo .env (que contiene credenciales), la carpeta /storage/logs y los artefactos de compilación de Vite.

12. Trabajo Pendiente

Las siguientes funcionalidades están diseñadas en la base de datos (modelos y migraciones creados) pero aún no tienen la capa de controladores y vistas completa:

- CRUD de Categorías en el panel de administración (CategoryController para admin).
- Filtrado del catálogo público por categoría.
- Asignación de categorías al crear/editar productos en el formulario de admin.
- Internacionalización (ES/EN): archivos de traducción en resources/lang/, selector de idioma en perfil.
- Gestión de perfil de usuario: cambio de contraseña, gestión de direcciones (CRUD), selector de idioma.
- Toggle de favoritos: FavoritoController con método store/destroy, botón en catálogo y detalle de producto.
- Vista de favoritos en el perfil del usuario.
- Ranking de productos más añadidos a favoritos en el panel de administración.
- Gestión de descuentos: interfaz para crear/asignar cupones de descuento a productos.
- Imágenes de producto: sistema de subida de imágenes con almacenamiento en storage/app/public.